

C++ Programming

Day 11: 연관 컨테이너와 컨테이너 어댑터

set, map, unordered_map, stack, queue

Contents

1	학습 목표	2
2	Day 11 개요	2
3	set	2
3.1	헤더와 선언	2
3.2	원소 삽입	3
3.3	자동 정렬과 이진 탐색 트리	3
3.4	탐색과 삭제	3
3.5	최종 코드	3
3.6	실습	4
4	map	5
4.1	헤더와 선언	5
4.2	대괄호 연산자로 삽입과 수정	5
4.3	first와 second	5
4.4	없는 키와 대괄호 연산자	5
4.5	최종 코드	5
4.6	실습	6
5	unordered_map	7
5.1	map과의 비교	7
5.2	사용 방법	7
5.3	최종 코드	7
5.4	실습	8
6	stack	9
6.1	주요 함수	9
6.2	pop의 특징	9
6.3	최종 코드	9
6.4	실습	10
7	queue	10
7.1	주요 함수	11
7.2	stack과 queue 비교	11
7.3	최종 코드	11
7.4	실습	12
8	컨테이너 선택 기준	12
9	실습 파일 구성	13

1. 학습 목표

학습 목표

이 장을 마치면 다음 내용을 설명하고 직접 코드로 작성할 수 있다.

- set의 중복 제거와 자동 정렬 특성 이해하기
- insert(), erase(), find()로 원소 관리하기
- 이진 탐색 트리의 기본 배치 원리 이해하기
- map에 키와 값을 한 쌍으로 저장하기
- first, second로 키와 값에 접근하기
- map의 대괄호 연산자가 없는 키를 생성할 수 있음을 이해하기
- map과 unordered_map의 정렬 및 탐색 방식 비교하기
- stack의 LIFO 구조와 top() 사용하기
- queue의 FIFO 구조와 front(), back() 사용하기
- 컨테이너 어댑터에서 pop()이 값을 반환하지 않는 이유 이해하기

2. Day 11 개요

Day 10에서는 vector와 반복자를 중심으로 순차 컨테이너를 학습했다. Day 11에서는 값을 정렬하거나 키로 검색하는 연관 컨테이너와, 데이터의 입출력 순서를 제한하는 컨테이너 어댑터를 학습한다.

자료구조	핵심 특징	대표 접근 방식
set	중복 없는 정렬된 값	값 탐색
map	정렬된 키와 값	키로 값 탐색
unordered_map	정렬되지 않은 키와 값	해시 기반 탐색
stack	나중에 넣은 값이 먼저 나옴	top()
queue	먼저 넣은 값이 먼저 나옴	front()

핵심 포인트

연관 컨테이너는 저장 위치보다 값이나 키를 중심으로 데이터를 관리한다. 컨테이너 어댑터는 내부 컨테이너의 기능을 제한하여 스택이나 큐의 동작만 제공한다.

3. set

set은 같은 자료형의 값을 저장하며 중복을 허용하지 않는 연관 컨테이너이다. 원소는 기본적으로 오름차순으로 정렬되어 유지된다.

3.1. 헤더와 선언

```

1 #include <set>
2
3 set<int> numbers;
4 set<string> names;
```

3.2. 원소 삽입

`insert()`로 원소를 추가한다. 이미 존재하는 값은 다시 저장되지 않는다.

```
1 set<int> numbers;
2
3 numbers.insert(5);
4 numbers.insert(2);
5 numbers.insert(8);
6 numbers.insert(2);
```

저장 결과는 다음과 같다.

2 5 8

3.3. 자동 정렬과 이진 탐색 트리

일반적인 이진 탐색 트리는 각 노드에서 작은 값은 왼쪽, 큰 값은 오른쪽에 배치한다.

```

      7
     / \
    4   9
   / \ / \
  2  5 8 10
```

이 구조를 왼쪽, 현재, 오른쪽 순서로 순회하면 값이 오름차순으로 나온다. 실제 표준은 내부 구현 방식을 강제하지 않지만, 많은 구현에서 `set`은 Red-Black Tree와 같은 균형 이진 탐색 트리를 사용한다. 따라서 삽입, 삭제, 탐색은 일반적으로 $O(\log n)$ 의 시간 복잡도를 가진다.

3.4. 탐색과 삭제

`find()`는 값을 찾으면 해당 원소의 반복자를 반환하고, 찾지 못하면 `end()`를 반환한다.

```
1 if (numbers.find(5) != numbers.end())
2 {
3     cout << "5 exists" << endl;
4 }
```

`erase()`에는 삭제할 값을 직접 전달할 수 있다.

```
1 numbers.erase(2);
```

3.5. 최종 코드

```
1 #include <iostream>
2 #include <set>
3
4 using namespace std;
5
6 int main()
7 {
8     set<int> numbers;
9
10    numbers.insert(5);
11    numbers.insert(2);
12    numbers.insert(8);
13    numbers.insert(2);
14    numbers.insert(1);
15 }
```

```
16 for (const auto& number : numbers)
17 {
18     cout << number << " ";
19 }
20
21 cout << endl;
22 cout << "Size: " << numbers.size() << endl;
23
24 if (numbers.find(5) != numbers.end())
25 {
26     cout << "5 exists" << endl;
27 }
28
29 numbers.erase(2);
30
31 for (const auto& number : numbers)
32 {
33     cout << number << " ";
34 }
35
36 cout << endl;
37
38 return 0;
39 }
```

3.6. 실습

실습

정수 7, 4, 7, 2, 4, 9를 set에 넣고 모든 원소를 출력한다. 중복이 제거되고 2 4 7 9가 출력되는지 확인한다.

```
1 #include <iostream>
2 #include <set>
3
4 using namespace std;
5
6 int main()
7 {
8     set<int> numbers;
9
10    numbers.insert(7);
11    numbers.insert(4);
12    numbers.insert(7);
13    numbers.insert(2);
14    numbers.insert(4);
15    numbers.insert(9);
16
17    for (const auto& number : numbers)
18    {
19        cout << number << " ";
20    }
21
22    cout << endl;
23
24    return 0;
25 }
```

4. map

map은 키와 값을 한 쌍으로 저장하는 연관 컨테이너이다. 각 키는 하나만 존재할 수 있으며, 값은 중복될 수 있다. 원소는 키를 기준으로 오름차순 정렬된다.

4.1. 헤더와 선언

```
1 #include <map>
2
3 map<string, int> scores;
```

첫 번째 타입은 키의 타입이고 두 번째 타입은 값의 타입이다.

4.2. 대괄호 연산자로 삽입과 수정

```
1 scores["Alice"] = 95;
2 scores["Bob"] = 87;
3 scores["Chris"] = 100;
```

같은 키에 다시 대입하면 새 원소가 추가되지 않고 기존 값이 수정된다.

```
1 scores["Alice"] = 98;
```

4.3. first와 second

map의 각 원소는 키와 값을 저장한 pair 형태이다. 반복문에서 first는 키, second는 값이다.

```
1 for (const auto& student : scores)
2 {
3     cout << student.first << " : "
4         << student.second << endl;
5 }
```

4.4. 없는 키와 대괄호 연산자

대괄호 연산자로 없는 키를 조회하면 해당 키와 기본값이 새로 삽입된다.

```
1 cout << scores["John"] << endl;
```

int의 기본값은 0이므로 위 코드는 "John" -> 0 원소를 생성한다. 존재 여부만 확인할 때는 find()를 사용하는 것이 안전하다.

```
1 if (scores.find("John") == scores.end())
2 {
3     cout << "John not found" << endl;
4 }
```

4.5. 최종 코드

```
1 #include <iostream>
2 #include <map>
3 #include <string>
4
5 using namespace std;
6
7 int main()
8 {
```

```

9  map<string, int> scores;
10
11  scores["Alice"] = 95;
12  scores["Bob"] = 87;
13  scores["Chris"] = 100;
14
15  cout << "Alice : " << scores["Alice"] << endl;
16  cout << "Bob : " << scores["Bob"] << endl;
17
18  cout << endl;
19
20  for (const auto& student : scores)
21  {
22      cout << student.first << " : "
23          << student.second << endl;
24  }
25
26  cout << endl;
27
28  scores.erase("Bob");
29
30  if (scores.find("Bob") == scores.end())
31  {
32      cout << "Bob not found" << endl;
33  }
34
35  return 0;
36 }

```

4.6. 실습

실습

학생 네 명의 이름과 나이를 map<string, int>에 저장한다. 모든 학생을 출력하고, Mike의 나이를 출력한 뒤 Jane을 삭제한다. Jane이 존재하지 않으면 Jane not found를 출력한다.

```

1  #include <iostream>
2  #include <map>
3  #include <string>
4
5  using namespace std;
6
7  int main()
8  {
9      map<string, int> students;
10
11      students["Tom"] = 20;
12      students["Jane"] = 22;
13      students["Mike"] = 21;
14      students["Lucy"] = 23;
15
16      for (const auto& student : students)
17      {
18          cout << student.first << " : "
19              << student.second << endl;
20      }
21
22      cout << students["Mike"] << endl;

```

```

23     students.erase("Jane");
24
25     if (students.find("Jane") == students.end())
26     {
27         cout << "Jane not found" << endl;
28     }
29
30     return 0;
31 }
32

```

5. unordered_map

`unordered_map`도 키와 값을 한 쌍으로 저장하지만, 키를 정렬하지 않는다. 일반적으로 해시 테이블을 사용하여 평균 $O(1)$ 의 삽입, 삭제, 탐색 성능을 제공한다.

5.1. map과의 비교

항목	map	unordered_map
원소 순서	키 기준 정렬	보장되지 않음
일반적 구현	균형 이진 탐색 트리	해시 테이블
탐색 복잡도	$O(\log n)$	평균 $O(1)$
최악의 탐색 복잡도	$O(\log n)$	$O(n)$
주요 목적	정렬된 데이터	빠른 키 검색

5.2. 사용 방법

사용 방식은 `map`과 거의 같다.

```

1 #include <unordered_map>
2
3 unordered_map<string, int> scores;
4
5 scores["Alice"] = 95;
6 scores.erase("Alice");

```

반복문으로 출력할 때 순서는 실행 환경이나 내부 상태에 따라 달라질 수 있다.

5.3. 최종 코드

```

1 #include <iostream>
2 #include <unordered_map>
3 #include <string>
4
5 using namespace std;
6
7 int main()
8 {
9     unordered_map<string, int> scores;
10
11     scores["Alice"] = 95;
12     scores["Bob"] = 87;
13     scores["Chris"] = 100;
14
15     cout << "Alice : " << scores["Alice"] << endl;
16     cout << endl;

```



```

17
18     for (const auto& student : scores)
19     {
20         cout << student.first << " : "
21             << student.second << endl;
22     }
23
24     cout << endl;
25
26     if (scores.find("Bob") != scores.end())
27     {
28         cout << "Bob exists" << endl;
29     }
30
31     scores.erase("Bob");
32
33     if (scores.find("Bob") == scores.end())
34     {
35         cout << "Bob deleted" << endl;
36     }
37
38     return 0;
39 }

```

5.4. 실습

실습

Apple, Banana, Orange, Grape의 가격을 unordered_map<string, int>에 저장한다. 모든 원소와 Orange의 가격을 출력한다. Banana를 삭제한 뒤 존재하지 않으면 Banana deleted를 출력한다.

```

1  #include <iostream>
2  #include <string>
3  #include <unordered_map>
4
5  using namespace std;
6
7  int main()
8  {
9      unordered_map<string, int> fruits;
10
11      fruits["Apple"] = 1200;
12      fruits["Banana"] = 800;
13      fruits["Orange"] = 1500;
14      fruits["Grape"] = 2000;
15
16      for (const auto& fruit : fruits)
17      {
18          cout << fruit.first << " : "
19              << fruit.second << endl;
20      }
21
22      cout << fruits["Orange"] << endl;
23
24      fruits.erase("Banana");
25
26      if (fruits.find("Banana") == fruits.end())

```

```

27 {
28     cout << "Banana deleted" << endl;
29 }
30
31 return 0;
32 }

```

6. stack

stack은 나중에 들어온 원소가 먼저 나오는 LIFO(Last In, First Out) 구조를 제공하는 컨테이너 어댑터이다.

Top
30
20
10

6.1. 주요 함수

함수	역할
push(value)	맨 위에 원소 추가
top()	맨 위 원소 접근
pop()	맨 위 원소 제거
size()	원소 개수 반환
empty()	비어 있는지 확인

6.2. pop의 특징

pop()은 제거한 값을 반환하지 않는다. 값을 저장하면서 제거하려면 먼저 top()을 호출한 뒤 pop()을 호출한다.

```

1 int value = numbers.top();
2 numbers.pop();

```

빈 스택에서 top()이나 pop()을 호출하면 올바른 동작이 보장되지 않으므로 먼저 empty()를 확인해야 한다.

6.3. 최종 코드

```

1 #include <iostream>
2 #include <stack>
3
4 using namespace std;
5
6 int main()
7 {
8     stack<int> numbers;
9
10    numbers.push(10);
11    numbers.push(20);
12    numbers.push(30);
13
14    cout << "Top : " << numbers.top() << endl;
15
16    numbers.pop();

```

```

17     cout << "Top : " << numbers.top() << endl;
18     cout << "Size : " << numbers.size() << endl;
19
20     while (!numbers.empty())
21     {
22         cout << numbers.top() << " ";
23         numbers.pop();
24     }
25
26     cout << endl;
27
28     return 0;
29 }
30

```

6.4. 실습

실습

정수 5, 10, 15, 20을 스택에 넣는다. 현재 맨 위 값을 출력하고 pop()을 두 번 호출한다. 남은 원소를 모두 출력한다.

```

1  #include <iostream>
2  #include <stack>
3
4  using namespace std;
5
6  int main()
7  {
8      stack<int> numbers;
9
10     numbers.push(5);
11     numbers.push(10);
12     numbers.push(15);
13     numbers.push(20);
14
15     cout << "Top : " << numbers.top() << endl;
16
17     numbers.pop();
18     numbers.pop();
19
20     while (!numbers.empty())
21     {
22         cout << numbers.top() << " ";
23         numbers.pop();
24     }
25
26     cout << endl;
27
28     return 0;
29 }

```

7. queue

queue는 먼저 들어온 원소가 먼저 나오는 FIFO(First In, First Out) 구조를 제공하는 컨테이너 어댑터이다.

Front Back
 100 200 300 400

7.1. 주요 함수

함수	역할
push(value)	뒤쪽에 원소 추가
front()	맨 앞 원소 접근
back()	맨 뒤 원소 접근
pop()	맨 앞 원소 제거
size()	원소 개수 반환
empty()	비어 있는지 확인

7.2. stack과 queue 비교

항목	stack	queue
출력 순서	LIFO	FIFO
원소 추가	push()	push()
원소 제거	pop()	pop()
접근 함수	top()	front(), back()
중간 원소 접근	불가능	불가능

7.3. 최종 코드

```

1 #include <iostream>
2 #include <queue>
3
4 using namespace std;
5
6 int main()
7 {
8     queue<int> numbers;
9
10    numbers.push(10);
11    numbers.push(20);
12    numbers.push(30);
13
14    cout << "Front : " << numbers.front() << endl;
15    cout << "Back : " << numbers.back() << endl;
16
17    numbers.pop();
18
19    cout << "Front : " << numbers.front() << endl;
20
21    while (!numbers.empty())
22    {
23        cout << numbers.front() << " ";
24        numbers.pop();
25    }
26
27    cout << endl;
28
29    return 0;
30 }
```

7.4. 실습

실습

정수 100, 200, 300, 400을 큐에 넣는다. 맨 앞과 맨 뒤 값을 출력한 뒤 pop()을 한 번 호출한다. 남은 모든 원소를 출력한다.

```

1 #include <iostream>
2 #include <queue>
3
4 using namespace std;
5
6 int main()
7 {
8     queue<int> numbers;
9
10    numbers.push(100);
11    numbers.push(200);
12    numbers.push(300);
13    numbers.push(400);
14
15    cout << numbers.front() << endl;
16    cout << numbers.back() << endl;
17
18    numbers.pop();
19
20    while (!numbers.empty())
21    {
22        cout << numbers.front() << " ";
23        numbers.pop();
24    }
25
26    cout << endl;
27
28    return 0;
29 }
```

8. 컨테이너 선택 기준

필요한 기능	적합한 컨테이너	이유
순서대로 저장하고 인덱스로 접근	vector	연속 저장과 빠른 임의 접근
중복 없는 정렬된 값 저장	set	자동 정렬과 유일한 원소
정렬된 키와 값 저장	map	키 기준 순서 유지
순서 없이 빠르게 키 검색	unordered_map	평균 상수 시간 탐색
최근 원소부터 처리	stack	LIFO 구조
먼저 들어온 원소부터 처리	queue	FIFO 구조

핵심 포인트

자료구조를 선택할 때는 저장할 데이터의 형태뿐 아니라 정렬 필요 여부, 중복 허용 여부, 접근 방식, 탐색 빈도를 함께 고려해야 한다.

9. 실습 파일 구성

실습

Day 11 파일은 다음과 같이 관리한다.

- 01_set.cpp
- 02_map.cpp
- 03_unordered_map.cpp
- 04_stack.cpp
- 05_queue.cpp
- practice/p01_set.cpp
- practice/p02_map.cpp
- practice/p03_unordered_map.cpp
- practice/p04_stack.cpp
- practice/p05_queue.cpp